软件构造：编写高质量的面向对象代码

Software Construction: Write high-quality object-oriented code

课程编号：22WHSE22006（2023版培养计划）

课程性质：专业基础课（3秋-必修）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋 xinguodong@163.com

课程难度：☆☆☆☆☆

QQ 群：软件构造2025

群　号：311528424

关键词1：软件构造 - 什么是软件构造?

- 软件工程知识体系指南 - SWEBOK（Guide to Software Engineering Body of Knowledge）由 IEEE（国际电气电子工程师协会）发布，旨在规范软件工程领域的知识体系框架。

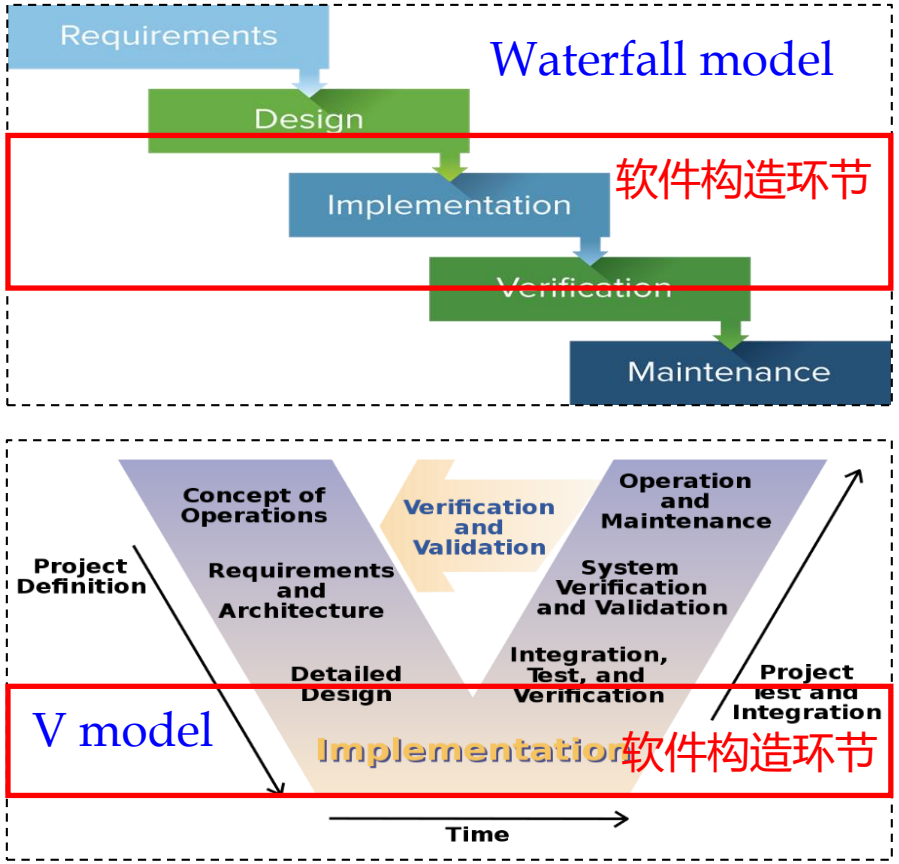


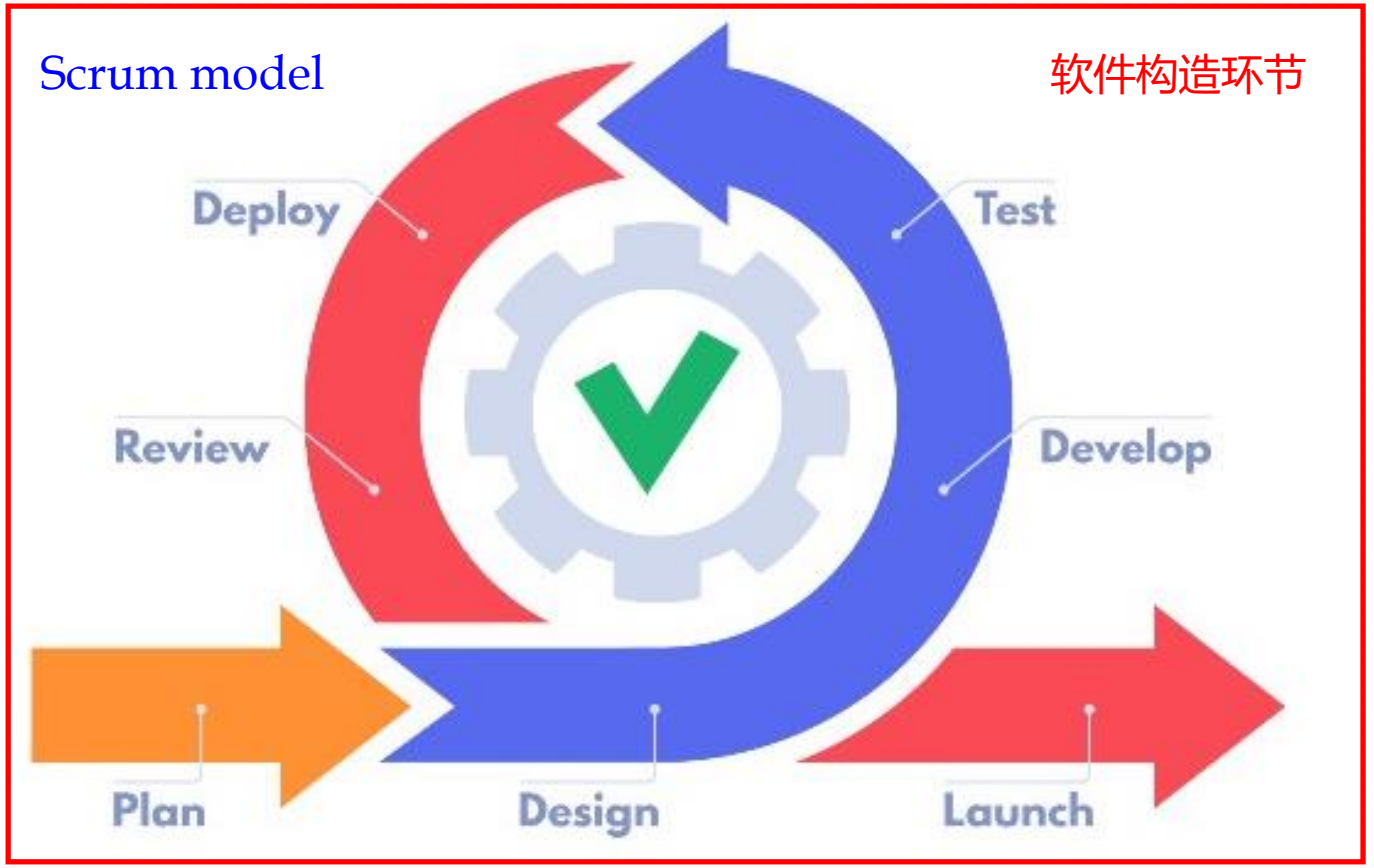
http://swebokwiki.org

V3 (2014)	V4 (2024)
Introduction	Introduction
1. Software Requirements	1. Software Requirements
	2. Software Architecture
2. Software Design	3. Software Design
3. Software Construction	4. Software Construction
4. Software Testing	5. Software Testing
	6. Software Engineering Operations
5. Software Maintenance	7. Software Maintenance
6. Software Configuration Management	8. Software Configuration Management
7. Software Engineering Management	9. Software Engineering Management
8. Software Engineering Process	10. Software Engineering Process
9. Software Engineering Models and Methods	11. Software Engineering Models and Methods
10. Software Quality	12. Software Quality
	13. Software Security
11. Software Engineering Professional Practice	14. Software Engineering Professional Practice
12. Software Engineering Economics	15. Software Engineering Economics
13. Computing Foundations	16. Computing Foundations
14. Mathematical Foundations	17. Mathematical Foundations
15. Engineering Foundations	18. Engineering Foundations
Appendix A. Knowledge Area Specifications	Appendix A. Knowledge Area Specifications
Appendix B. Standards	Appendix B. Standards
Appendix C. Consolidated Reference List	Appendix C. Consolidated Reference List

在 SWEBOK中，软件构造划分成狭义的软件构造和广义的软件构造

关键词1：软件构造 - 什么是软件构造?





狭义的软件构造：瀑布模型，V模型，软件构造主要是指实现 implementation，往上包括部分设计，往下涵盖部分单元测试

广义的软件构造：例如在迭代过程模型 Scrum，软件构造与其他步骤同时发生或重叠，将软件构造当成这些活动的集合体。

本课程内容限定于狭义的软件构造。

关键词2：高质量 - 如何评价代码质量的高低?

鲁棒性 (robustness)	直接 (straightforward)	正确性 (correctness、bug free)	易用性 (usability)	整洁 (clean)
健壮性 (robustness)	可扩展性 (extensibility)	可测试性 (testability)	分层清晰 (well-layered)	
可用性 (reliability)	可维护性 (maintainability)	模块化 (modularity)	清晰 (clarity)	简单 (simple)
可伸缩性 (scalability)	可读性 (readability)	高内聚低耦合 (high cohesion loose coupling)		
稳定性 (stability)	可理解性 (understandability)	高效 (high efficiency)	优雅 (elegant)	好 (good)
兼容性 (compatibility)	易修改性 (changeability)	高性能 (high performance)	少即是多 (less code is more)	
灵活性 (flexibility)	可复用 (reusability)	安全性 (security)	文档详尽 (well documented)	

关键词2：高质量 - 如何评价代码质量的高低?

这些词汇从不同维度说明代码质量，并不能通过单一的维度去评价一段代码写的好坏。
例如，一段代码的可扩展性很好，但可读性很差，不能说这段代码质量高。

鲁棒性 (robustness)	直接 (straightforward)	正确性 (correctness、bug free)	易用性 (usability)	整洁 (clean)
健壮性 (robustness)	可扩展性 (extensibility)	可测试性 (testability)	分层清晰 (well-layered)	
可用性 (reliability)	可维护性 (maintainability)	模块化 (modularity)	清晰 (clarity)	简单 (simple)
可伸缩性 (scalability)	可读性 (readability)	高内聚低耦合 (high cohesion loose coupling)		
稳定性 (stability)	可理解性 (understandability)	高效 (high efficiency)	优雅 (elegant)	好 (good)
兼容性 (compatibility)	易修改性 (changeability)	高性能 (high performance)	少即是多 (less code is more)	
灵活性 (flexibility)	可复用 (reusability)	安全性 (security)	文档详尽 (well documented)	

除此之外，不同的评价维度也并不是完全独立，有些是具有包含关系、重叠关系或者互相影响。

例如，代码的可读性好、可扩展性好，就意味着代码的可维护性好。

结论：代码质量评价的主观性，使得这种主观评价的准确度，跟工程师自身经验有极大的关系

关键词2：高质量 - 如何评价代码质量的高低?

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
1	可维护性 (maintainability)	在不破坏原有代码设计、不引入新的 bug 的情况下，能够快速地修改或者添加代码。强调系统在修改和新增功能时的效率和安全性

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
2	可读性 (readability)	code review 是一个很好的测验代码可读性的手段。你的同事可以轻松地读懂你写的代码，那说明你的代码可读性很好。

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
3	可扩展性 (extensibility)	不修改或少量修改原有代码的情况下，通过扩展的方式添加新的功能代码。强调系统在面对业务需求变化时的适应能力。

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
4	灵活性 (flexibility)	代码能够适应多种调用方式或需求变化的能力，表现为在不修改原有代码的前提下，通过扩展或配置实现新功能或适应不同场景。易扩展、易复用或者易用。

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
5	简洁性 (simplicity)	尽量保持代码简单。代码简单、逻辑清晰，也就意味着易读、易维护。编写代码的时候，往往也会把简单、清晰放到首位

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
6	可复用性 (reusability)	尽量减少重复代码的编写，复用已有的代码。可复用性是很多设计原则、思想、模式等所要达到的最终效果

关键词2：高质量 - 如何评价代码质量的高低?

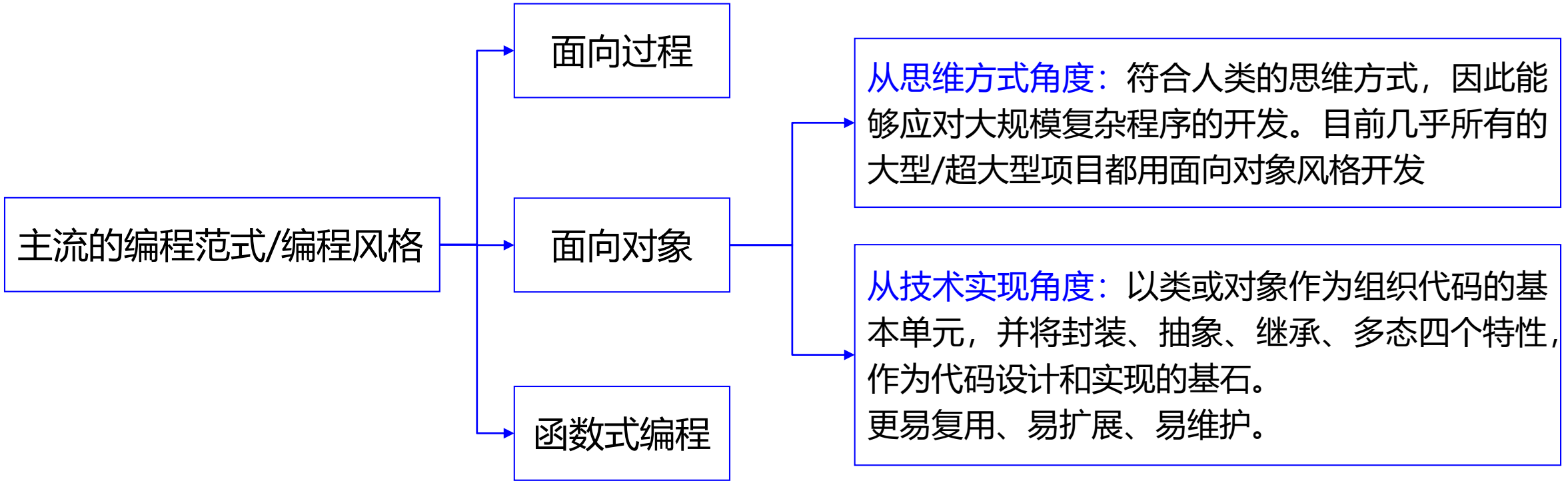
No.	最常用的评价标准	含义
7	可测试性 (testability)	程序被测试的难易程度。测试性能从侧面上非常准确地反应代码质量的好坏。 代码的可测试性差，比较难写单元测试，那基本上说明代码设计得有问题

关键词2：高质量 - 如何评价代码质量的高低?

No.	最常用的评价标准	含义
1	可维护性 (maintainability)	在不破坏原有代码设计、不引入新的 bug 的情况下，能够快速地修改或者添加代码。强调系统在修改和新增功能时的效率和安全性
2	可读性 (readability)	code review 是一个很好的测验代码可读性的手段。你的同事可以轻松地读懂你写的代码，那说明你的代码可读性很好。
3	可扩展性 (extensibility)	不修改或少量修改原有代码的情况下，通过扩展的方式添加新的功能代码。强调系统在面对业务需求变化时的适应能力。
4	灵活性 (flexibility)	代码能够适应多种调用方式或需求变化的能力，表现为在不修改原有代码的前提下，通过扩展或配置实现新功能或适应不同场景。易扩展、易复用或者易用。
5	简洁性 (simplicity)	尽量保持代码简单。代码简单、逻辑清晰，也就意味着易读、易维护。编写代码的时候，往往也会把简单、清晰放到首位
6	可复用性 (reusability)	尽量减少重复代码的编写，复用已有的代码。可复用性是很多设计原则、思想、模式等所要达到的最终效果
7	可测试性 (testability)	程序被测试的难易程度。测试性能从侧面上非常准确地反应代码质量的好坏。代码的可测试性差，比较难写单元测试，那基本上说明代码设计得有问题

不同的评价维度并不是完全独立，有些是具有包含关系、重叠关系或者互相影响。

关键词3：面向对象


主流的编程范式/编程风格
面向过程
面向对象
函数式编程
从思维方式角度：符合人类的思维方式，因此能够应对大规模复杂程序的开发。目前几乎所有的大型/超大型项目都用面向对象风格开发
从技术实现角度：以类或对象作为组织代码的基本单元，并将封装、抽象、继承、多态四个特性，作为代码设计和实现的基石。
更易复用、易扩展、易维护。

如何写出高质量的代码？ – 编写高质量代码的方法论

面向对象

- 封装、抽象、继承、多态
- 面向对象编程 vs 面向过程编程
- 面向对象分析、设计、编程
- 接口 vs 抽象类
- 基于接口而非实现编程
- 多用组合，少用继承
- 贫血模型和充血模型

基础

设计原则

- SRP单一职责
- OCP开闭原则
- LSP里氏代换原则
- ISP接口隔离原则
- DIP依赖倒置原则

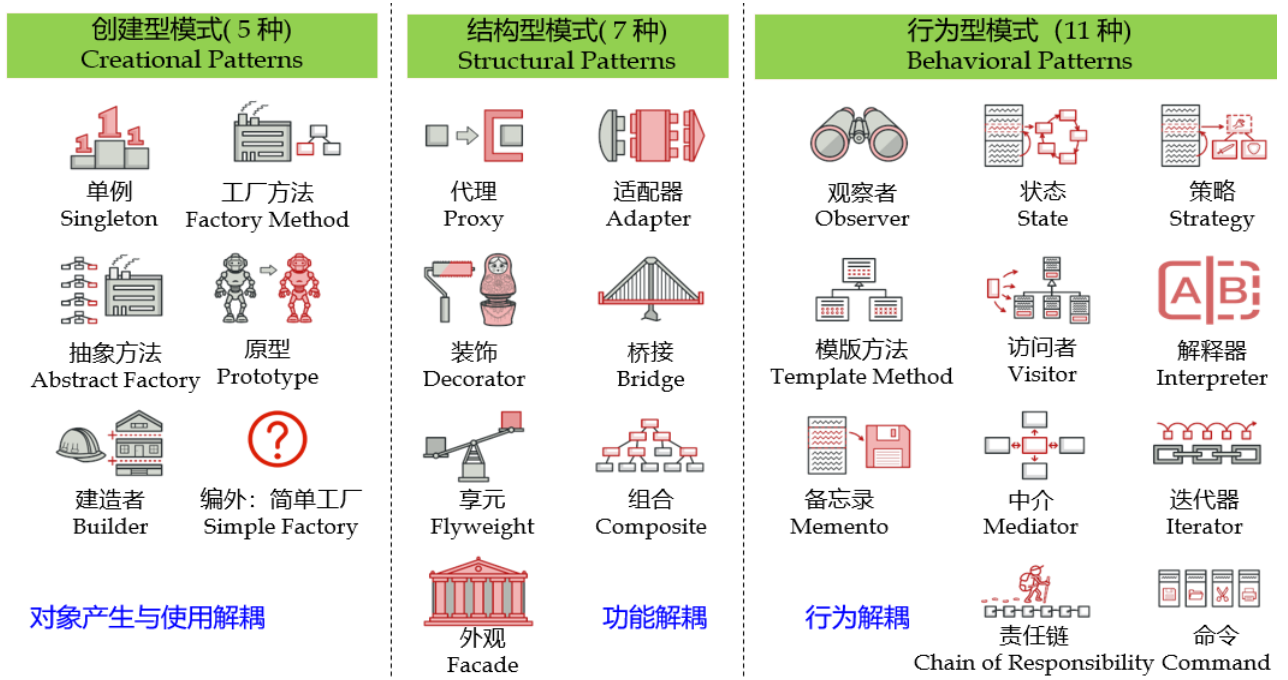
SOLID

- DRY、KISS、YAGNI
- LOD法则

内功

设计模式

招式



开发规范：编码规范、异常日志规范等

规矩

重构：四要素、单元测试与代码的可测试性等

修炼

课程内容

① 面向对象(课堂 6学时)
第 1-3 章

② 设计原则(课堂 4 学时)
第 4-5 章，实验 1（实验 4 学时）

③ 设计模式（课堂 14 学时）
第 6-8 章，实验 2-4（实验 12 学时 ）

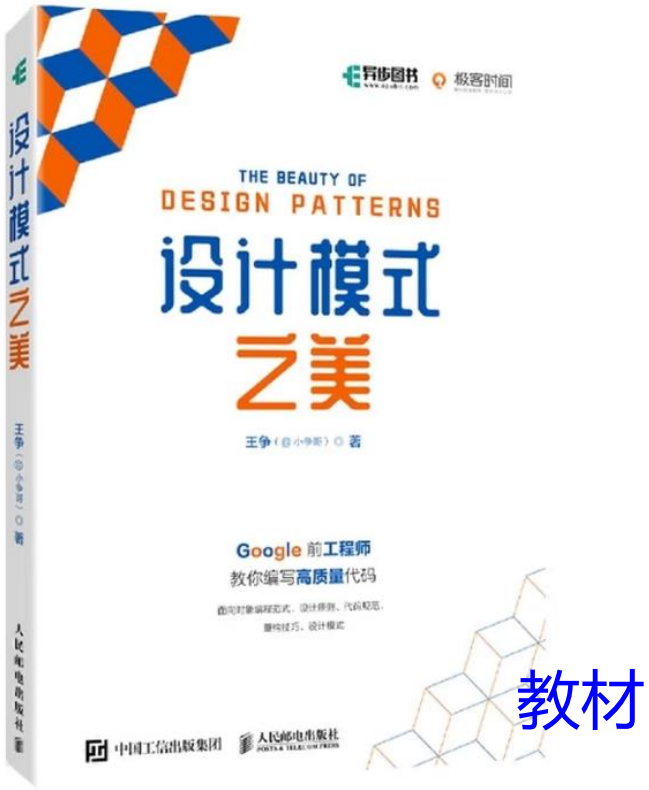
④ 开发规范（课堂4学时）第9章

⑤ 重构（课堂4学时）第10章

	序号	方式	授课内容	学时
基础	1	讲课	第 1 章 类、类间关系的UML	2
	2	讲课	第 2 章 面向对象特征及其解决的问题	2
	3	习题	第 3 章 面向对象编程习题课	2
内功	4	讲课	第 4 章 SOLID	2
	5	讲课	第 5 章 其他原则	2
	6	实验	实验1：开发基础（注解、反射）	4
招式	7	讲课	第 6 章 创建型设计模式	6
	8	实验	实验2：Bean容器构建（IOC）	4
	9	讲课	第 7 章 结构型设计模式	4
	10	实验	实验3：对象的自动装配（DI）	4
	11	讲课	第 8 章 行为型设计模式	4
	12	实验	实验4：重构	4
规矩	13	讲课	第 9 章 代码风格	4
修炼	14	讲课	第10章 重构	4

课堂 32 学时/综合实验 16 学时。第10-17 周，11月04日~12月25日

教材 & 参考资料



教材

作者：王争
出版社：人民邮电出版社
ISBN：9787115584748
出版时间：2022-06-01



作者：(美) Eric Freeman 等
出版社：东南大学出版社
ISBN：9787564199715
出版时间：2022-03-01



作者：(美)马丁. 福勒(Martin Fowler)
出版社： 人民邮电出版社
ISBN：9787115508645
出版时间：2019-04-01

成绩考核

- ① 课堂测验，占总成绩10%：在线，不定期不定量课堂测验，累计分数折算成总成绩的 10%
- ② 四次实验，占总成绩20%：实验主题：基于注解的IOC容器设计与实现。分成四个实验，共16学时，每次4学时，独立完成；拟采用验证实验方式进行，当堂给分（1/0）+ 实验报告。
  - 实验1：环境搭建及开发基础
  - 实验2：实现 IOC（Inversion of Control） 功能
  - 实验3：实现 DI（Dependency Injection） 功能
  - 实验4：重构
- ③ 期末考试，占总成绩70%：闭卷。卷面100分，折合成总分的70%，拟采用试题类型包括：
  - 客观题30%：选择/判断
  - 主观题70%：程序改错/简答20%、重构20%、设计30%（大致分数分配）

Lec01：类、类之间的关系的 UML 表示

UML representation of classes and relationships between classes

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

Email：xinguodong@163.com



QQ 群：软件构造2025

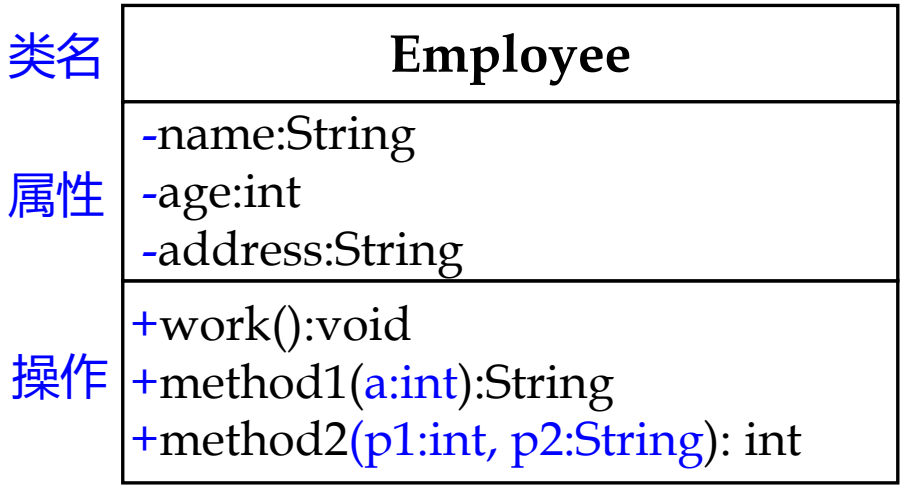
群　号：311528424

1. 类的 UML 表示

UML representation of class

1.1 类名、属性操作及其可见性

- UML采用矩形框表示类，划分为三个区域，分别表示类名、属性和操作



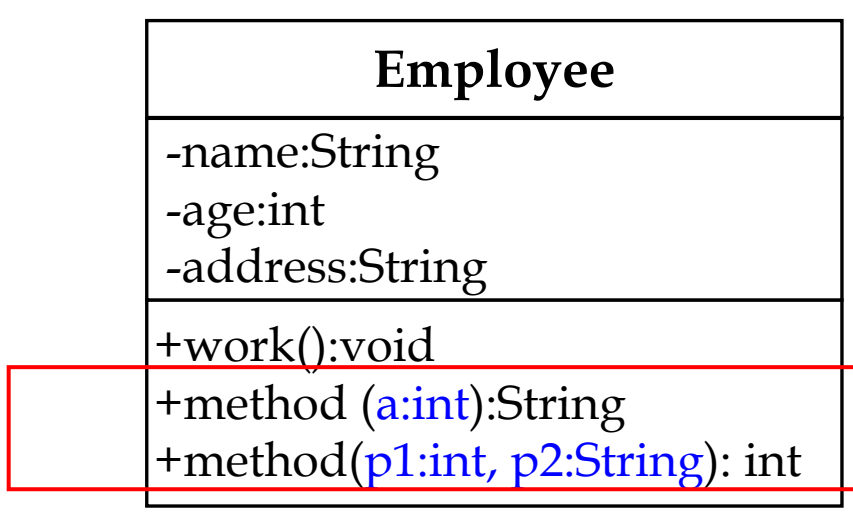

```
public class Employee {
    private String name;
    private int age;
    private String address;
    public void work(){  //…     }
    public String method1(int a){   //…   }
    public int method2(int p1,String p2){ //… }
}
```

可见性修饰符		当前类	同包	其他包
+	public	√	√	√
#	protected	√	√	×
~	default	√	√	×
-	private	√	×	×

备注：代码及可见性修饰符均以 Java 语言为例

1.2 方法重载 overloading

- 方法重载 overloading 是指在一个类中定义多个同名的方法，但要求每个方法具有不同的参数的类型或参数的个数。



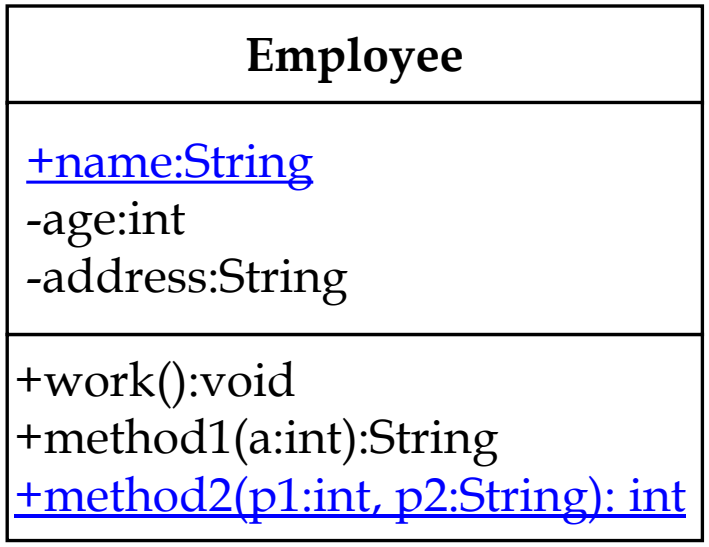

```
public class Employee {
    private String name;
    private int age;
    private String address;

    public void work(){  }
    public String method(int a){ … }
    public int method(int p1,String p2){…}
}
```

- 方法重载的判断标准：仅取决于参数列表的差异；与返回类型、访问修饰符、异常声明以及参数名称无关

1.3 静态变量和静态方法

- 类中可以定义静态变量和静态方法。静态变量称为类变量 Class variable； 静态方法就变成了类方法 class methods。UML中用下划线表示。

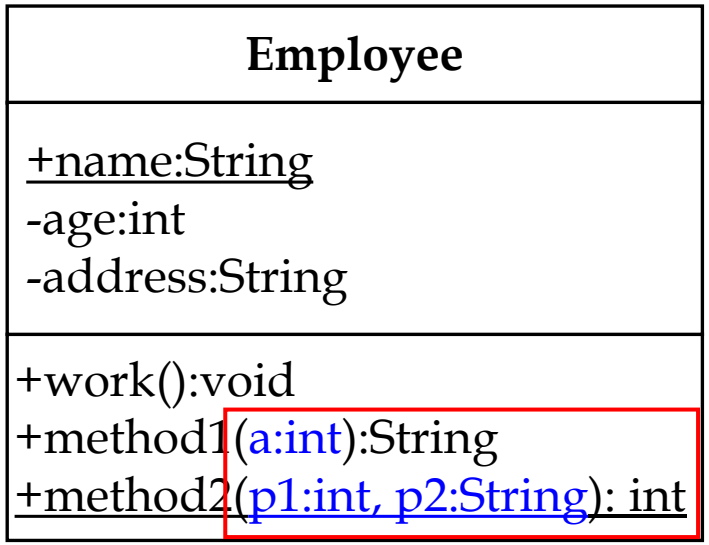



```
public class Employee {
    public static String name;
    private int age;
    private String address;

    public void work(){  }
    public String method1(int a){ … }
    public static int method2(int p1,String p2){…}
}
```

1.4 参数的表示

- 属性，返回值，方法中的参数，均是先写名称，再写类型。名称与类型之间用半角冒号“:”间隔



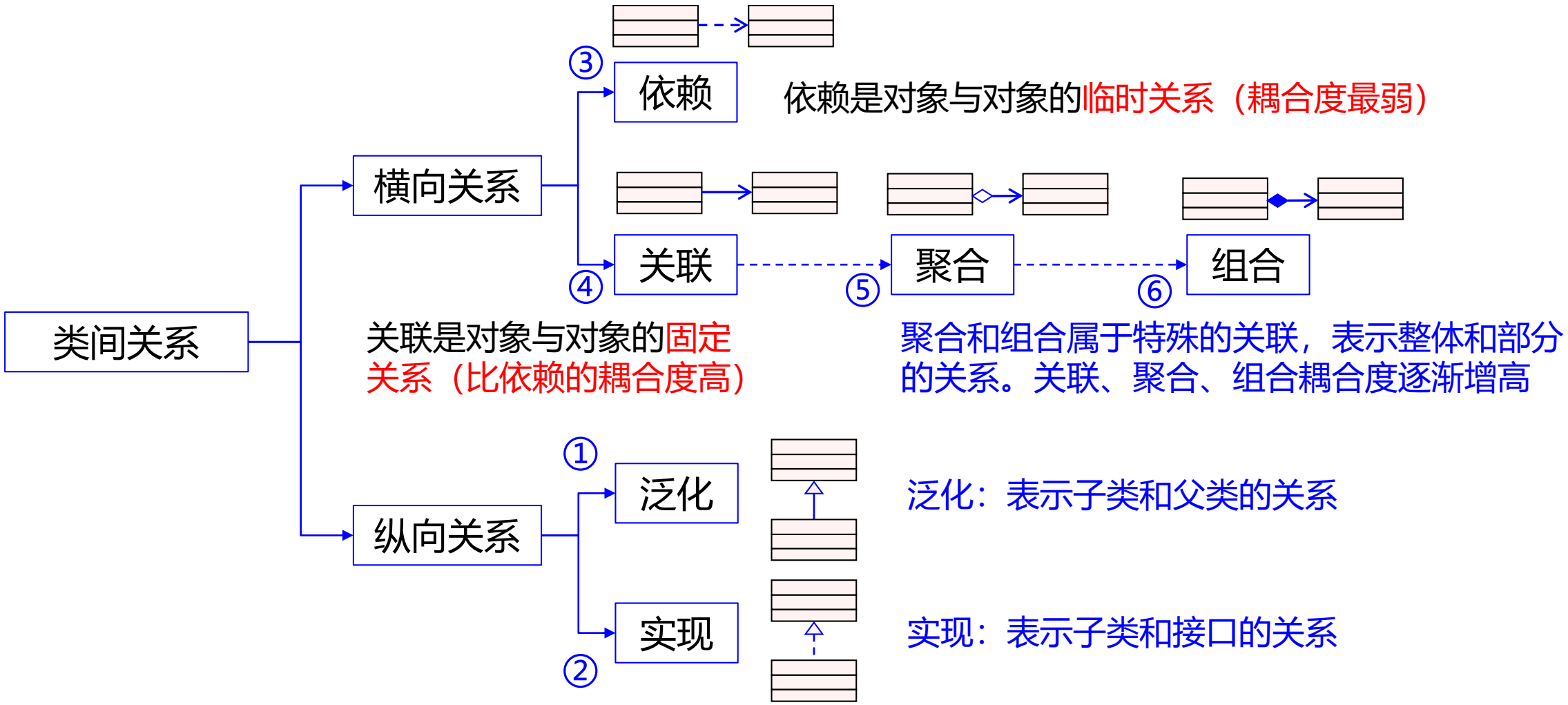

```
public class Employee {
    public static String name;
    private int age;
    private String address;

    public void work(){  }
    public String method1(int a){ … }
    public static int method2(int p1,String p2){…}
}
```

2. 类之间六种关系的UML表达

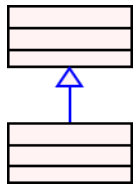
UML representation of six relationships between classes

2.1 类与类之间的关系汇总

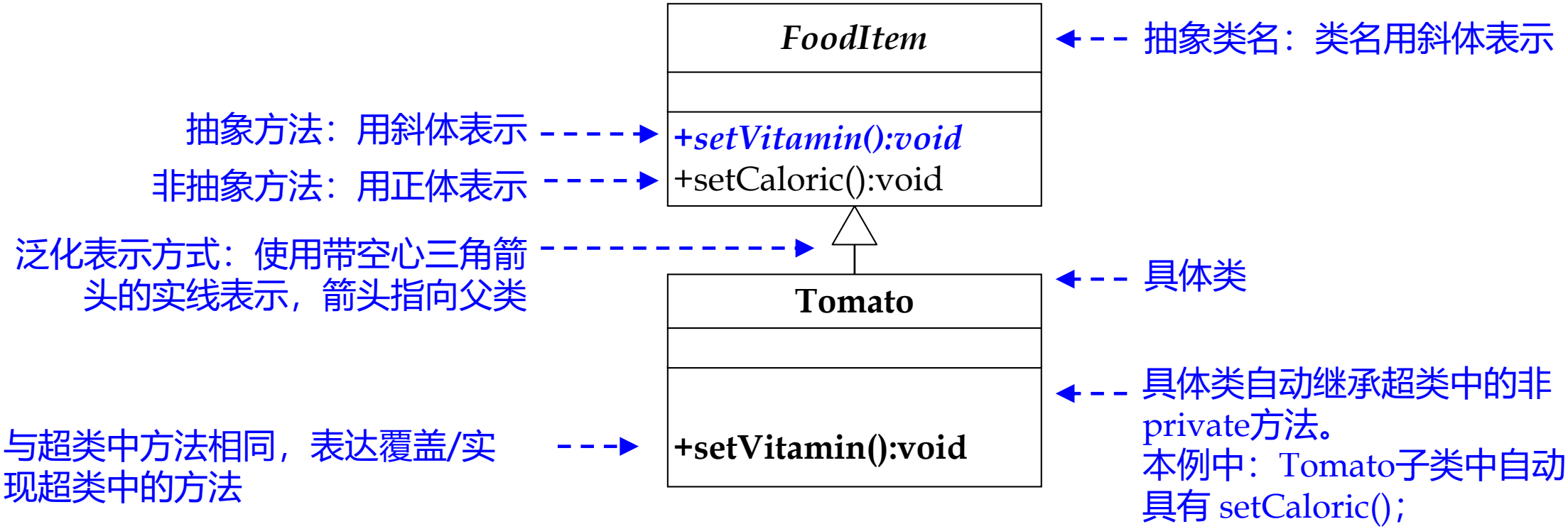


耦合强度：横向关系耦合度较低，纵向关系耦合度较高；依赖<关联<聚合<组合<泛化（继承）

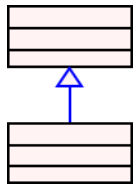
2.1 垂直关系：① - 泛化关系



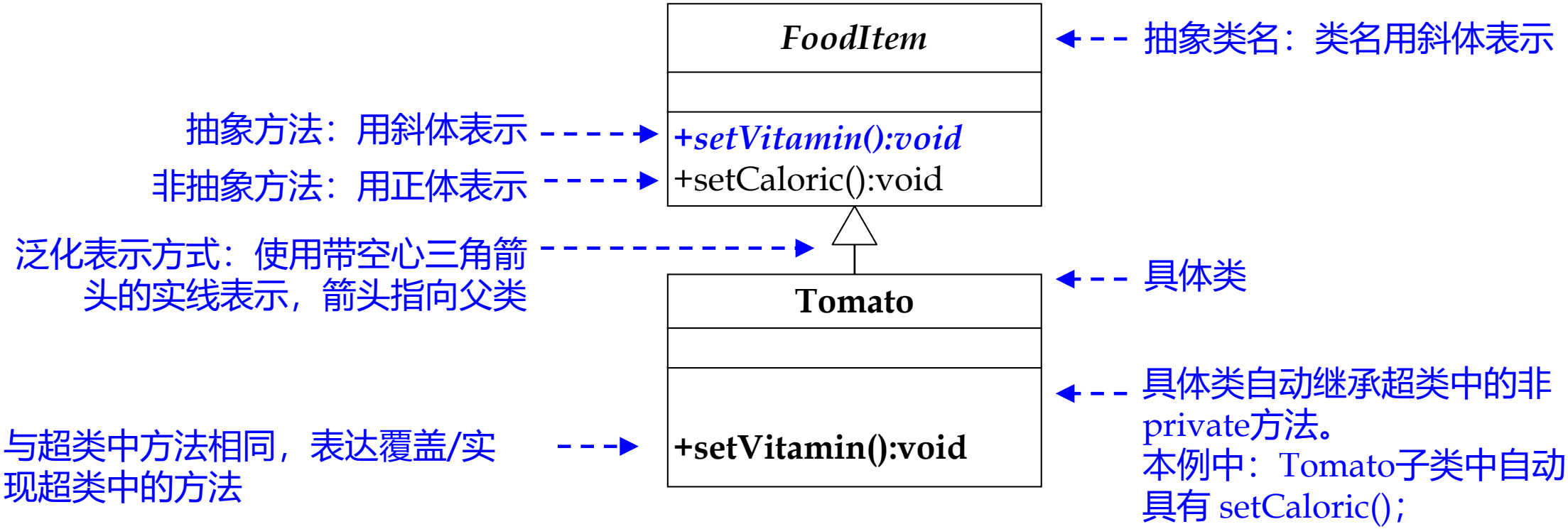
- 泛化关系：类与类之间/接口与接口之间的一种继承关系，表示一般与特殊的关系，指定子类如何特化父类的所有特征和行为



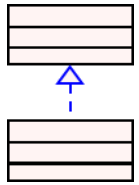
2.1 垂直关系：① - 泛化关系



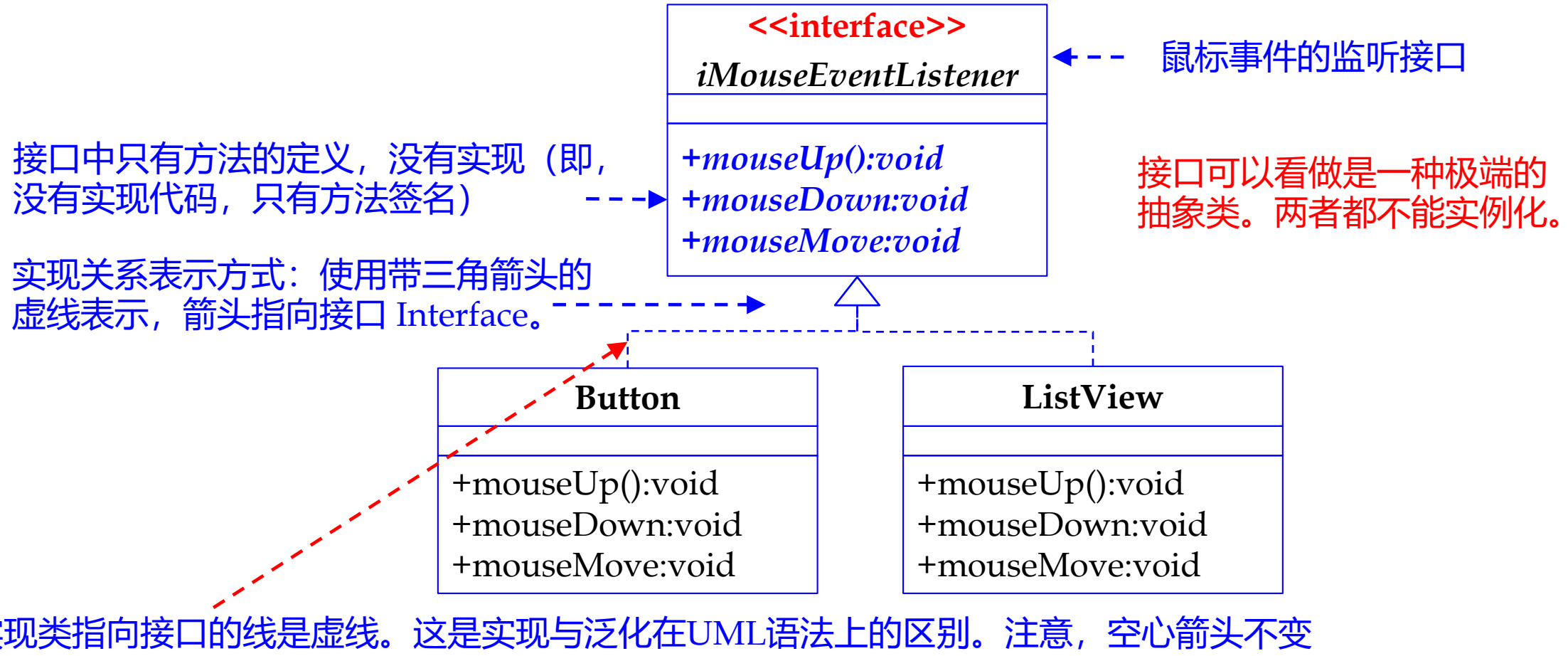
- 泛化关系：类与类之间/接口与接口之间的一种继承关系，表示一般与特殊的关系，指定子类如何特化父类的所有特征和行为



2.1 垂直关系：② - 实现关系

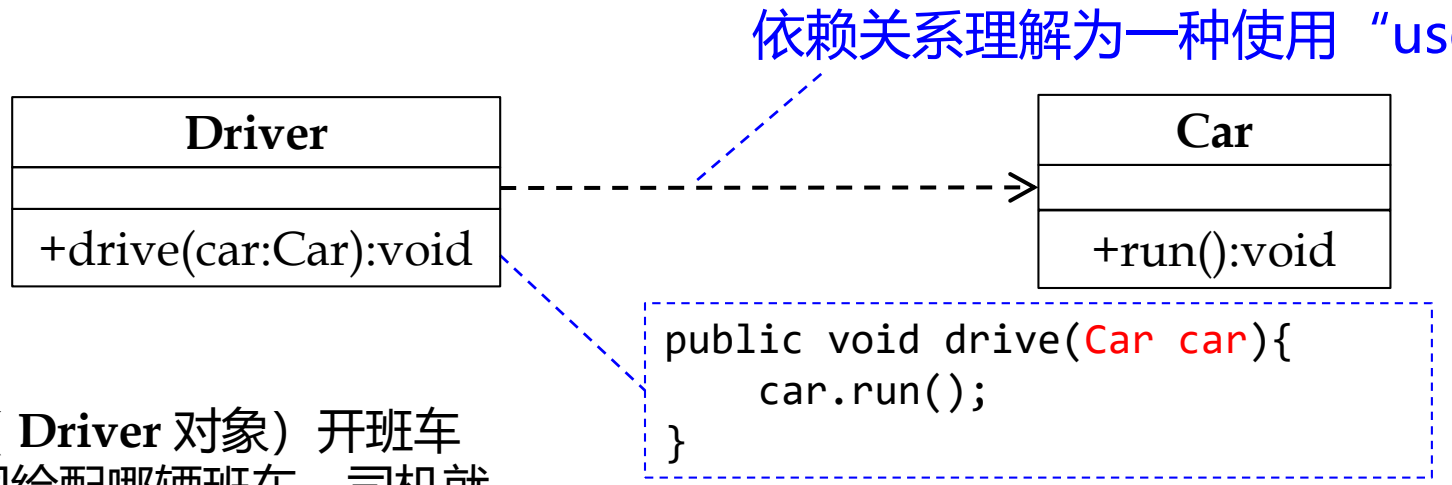


- 实现关系也是一种继承关系，子类（实现类）实现接口 Interface 的所有行为（即操作）



2.2 水平关系：③ - 依赖关系

- 两个相对独立的对象，当一个对象负责构造另一个对象的实例，或依赖另一个对象的服务时，这两个对象之间主要体现为依赖关系

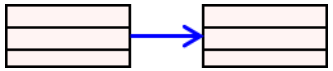


如上图，班车司机（Driver 对象）开班车（Car 对象），公司给配哪辆班车，司机就使用哪辆班车

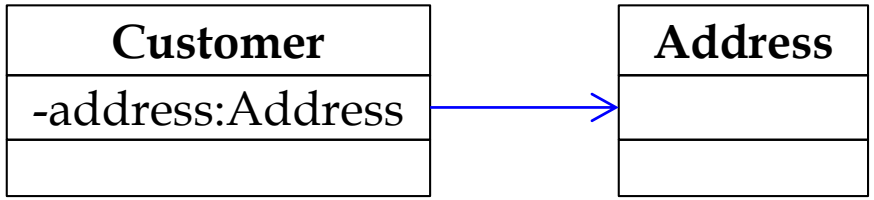
类A的对象 使用到了类B的对象，并且这种使用关系具有偶然性，临时性，是非常弱的关系，但是B类对象中的变化会影响到类A对象

依赖关系常见的三种方式：

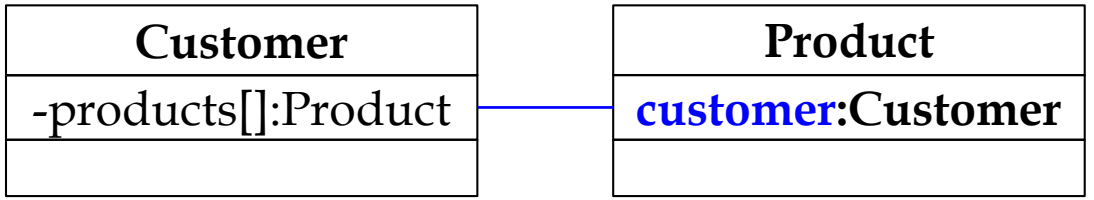
- ✓ 某个类的方法使用另一个类的对象作为参数（如上图）
- ✓ 在一个类的方法中将另一个类的对象作为其局部变量
- ✓ 在一个类的方法中调用另一个类的静态方法

2.2 水平关系：④ - 关联关系 

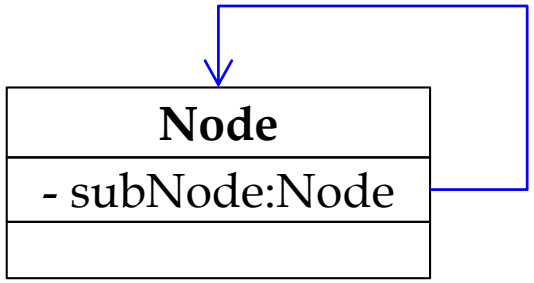
- 对于两个相对独立的对象，当一个对象与另一个对象存在固定的对应关系时，这两个对象之间为关联关系。



（1）单向关联：类 A 当中使用了类 B，其中类 B 是作为类 A 的成员变量。使用带普通箭头的实线表示，指向被拥有者



（2）双向关联： 类 A 当中使用了类 B 作为成员变量，同时类 B 中也使用了类 A 作为成员变量。可以使用有两个箭头或者没有箭头实线表示

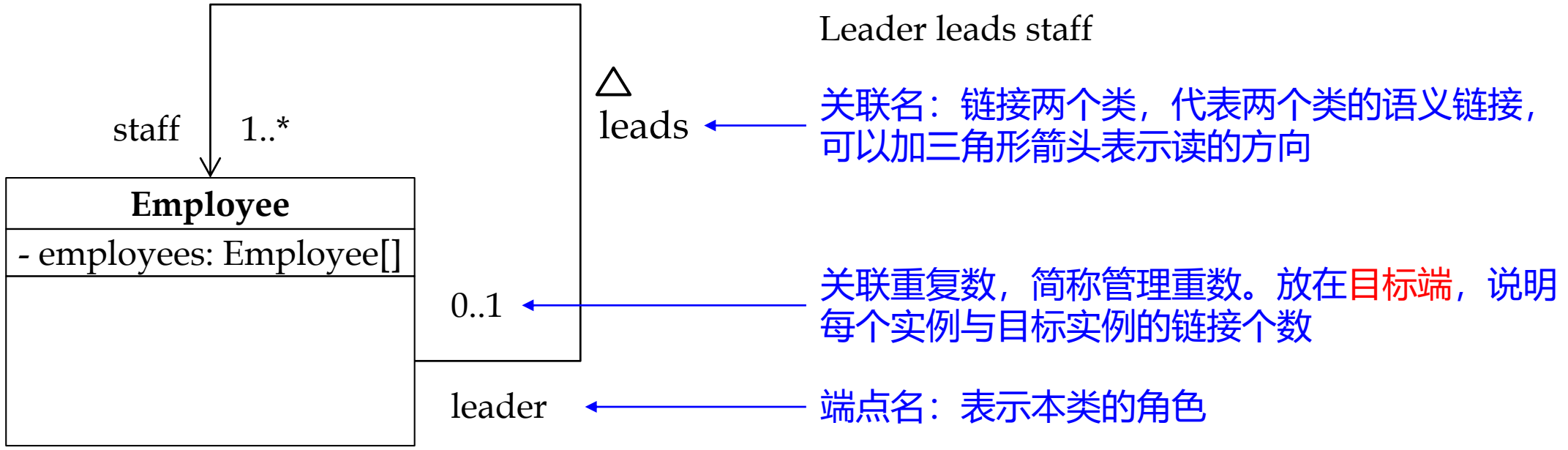


（3）自关联：自己包含自己

关联关系是两个类一种强的依赖关系，与依赖关系中的偶然性不同，关联关系是长期性的。可以理解为一种拥有或者对应关系，称作为“Has-a”的关系

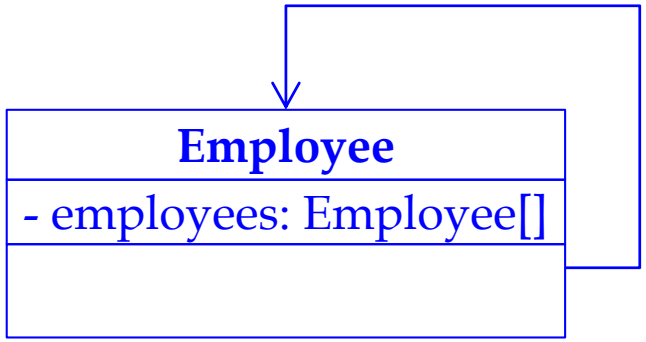
2.2 水平关系：④ - 关联关系

- 关联名、端点名（角色）及多重性



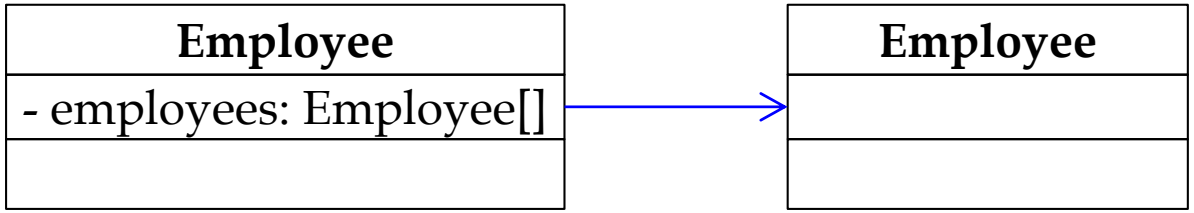
表达的语义：staff 员工和 leader 领导都是雇员；一个 leader 可以领导 lead 一到多个员工，一个员工可以没有领导，也可以有一个领导

2.2 水平关系：④ - 关联关系

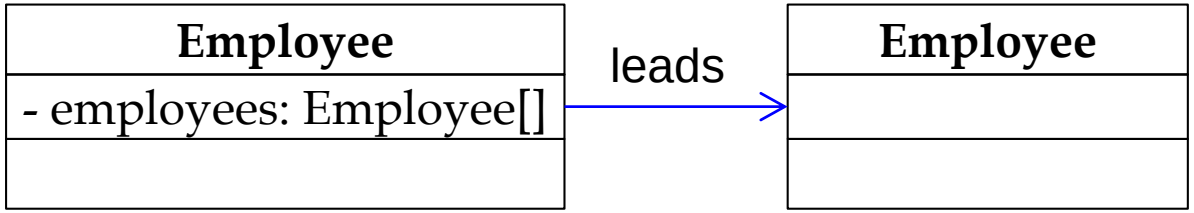


自关联比较难理解，可以将理解过程分成三步：

①发现语义不明确，因此可以将其展开如下：



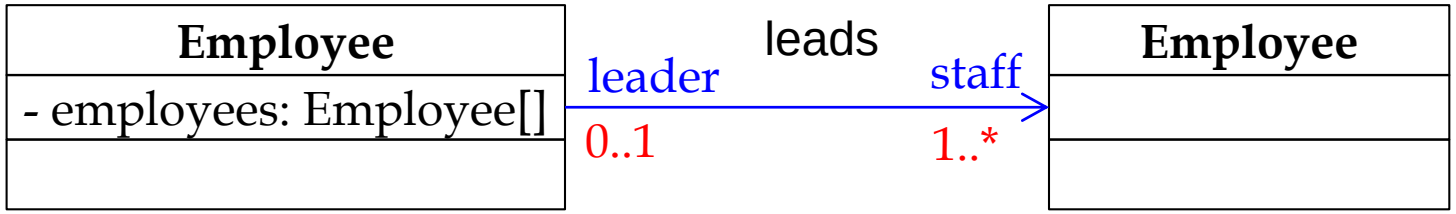
②发现语义还不明确。加上关联名 leads，语义上变成了：员工领导员工。如下：



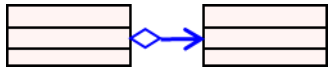
多重性表达表（n 可以表示成 *）

0	恰好为0
0..1	0或者1
0..n	0或者多个
1	恰好为1
1..n	1或多个
n	n或更多

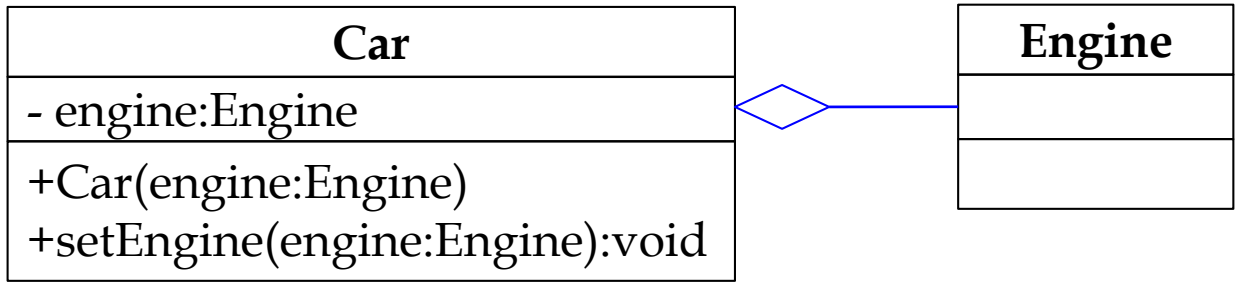
③进一步明确语义：在关联线两端加角色名和关联重数。



关联重数放在目标端，说明每个实例与目标实例的链接个数。③的语义：一个 leader，可以领导 leads 1 到多个员工；一个 staff，被 0 或 1 个 leader 领导。

2.2 水平关系：⑤ - 聚合关系 

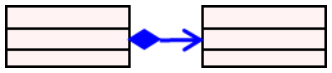
- 聚合关系的对象之间存在着包容关系，聚合关系强调是“整体”包含“部分”，但是“部分”可以脱离“整体”而单独存在。



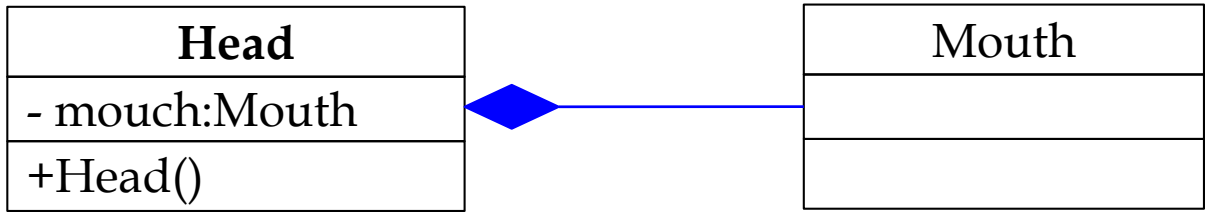
表示方式：带空心菱形的实心线，菱形指向整体

  - 汽车包含了发动机，汽车是整体，发动机是汽车的一部分；发动机脱离了汽车也能单独存在，可以生成发动机对象。

- 聚合关系属于关联关系的一种，耦合度强于关联

2.2 水平关系：⑥ - 组合关系 

- 组合关系是比聚合关系还要强的关系，它要求普通的聚合关系中代表整体的对象负责代表部分的对象的生命周期。
  - 即“整体”负责“部分”的生命周期，它们之间是共生共死的，并且“部分”单独存在时没有任何意义，例如：



表示方法：带实心菱形的实线，菱形指向整体。
以上类图表达：嘴 Mouth 是头 Head 的一部分，且不能脱离了头而单独存在

- 关联、聚合和组合的代码实现是相同的，仅仅是在语义上有所区别。

3. 练习：根据类图编写代码框架

Write code framework based on class diagram

3.1 需求描述

- 为汽车厂家设计一套驱动车载音响的协议（CarAudio），让凡是遵守这个协议的车载音响制造商所生产的产品，都能在汽车（Car）上使用。车载音响上的功能包括：
  - 播放（play）
  - 暂停（pause）
  - 上一首（prev）
  - 下一首（next）
  - 调节声音大（turnUp）
  - 调节声音小（turnDown）。
- 目前有 Philips 和 BOSE 两个品牌满足这个协议。
- 绘制类图，并实现代码框架

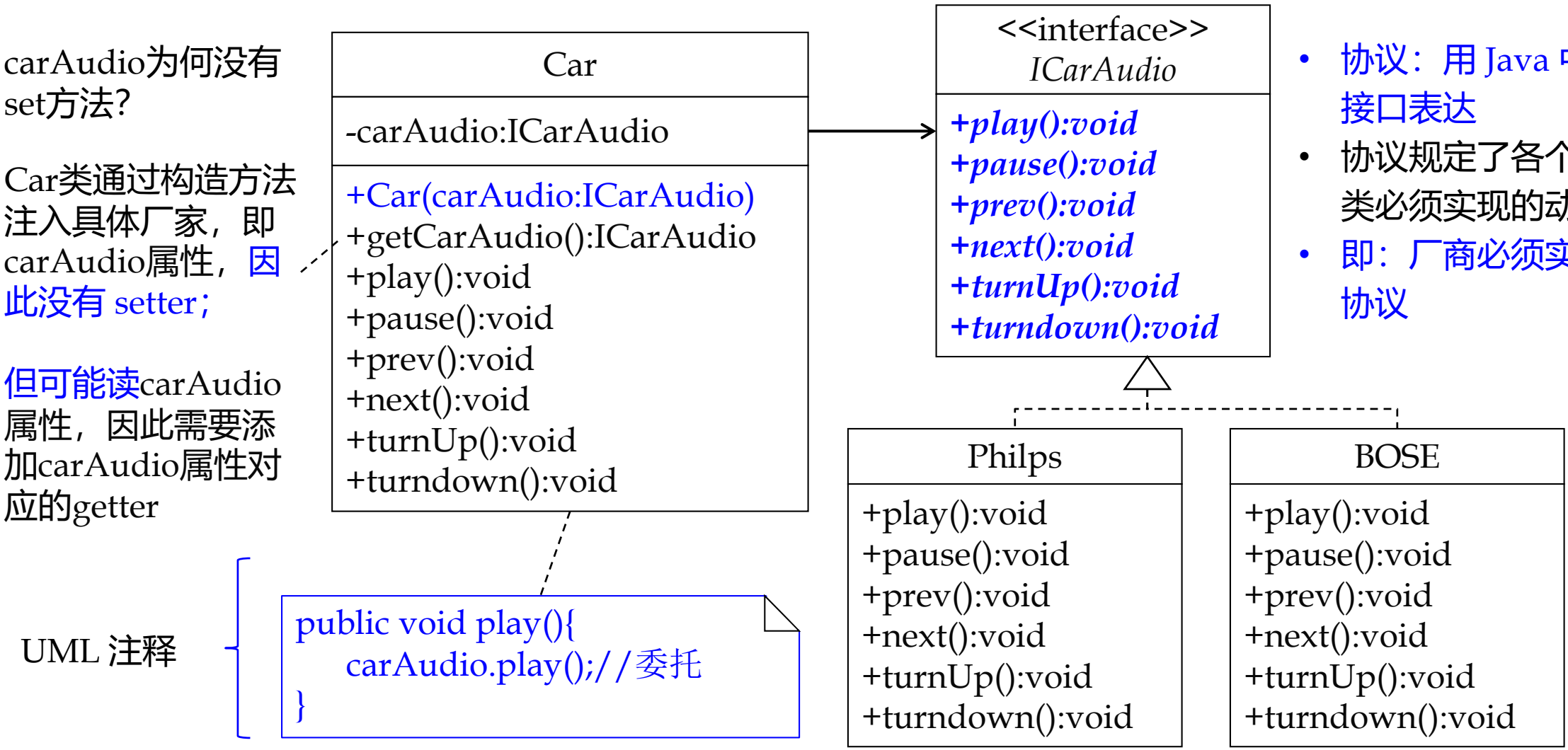
- ✓ 所谓协议，在面向对象的语言中都是指接口 Interface
- ✓ 协议规定了各个音箱厂商必须实现的动作（即方法method），但没有具体实现
- ✓ 即：只给出了方法的定义（返回值，方法名，入参，方法的注释文档，说明方法的作用），方法的实现由厂商负责

3.2 类图设计

carAudio为何没有set方法?

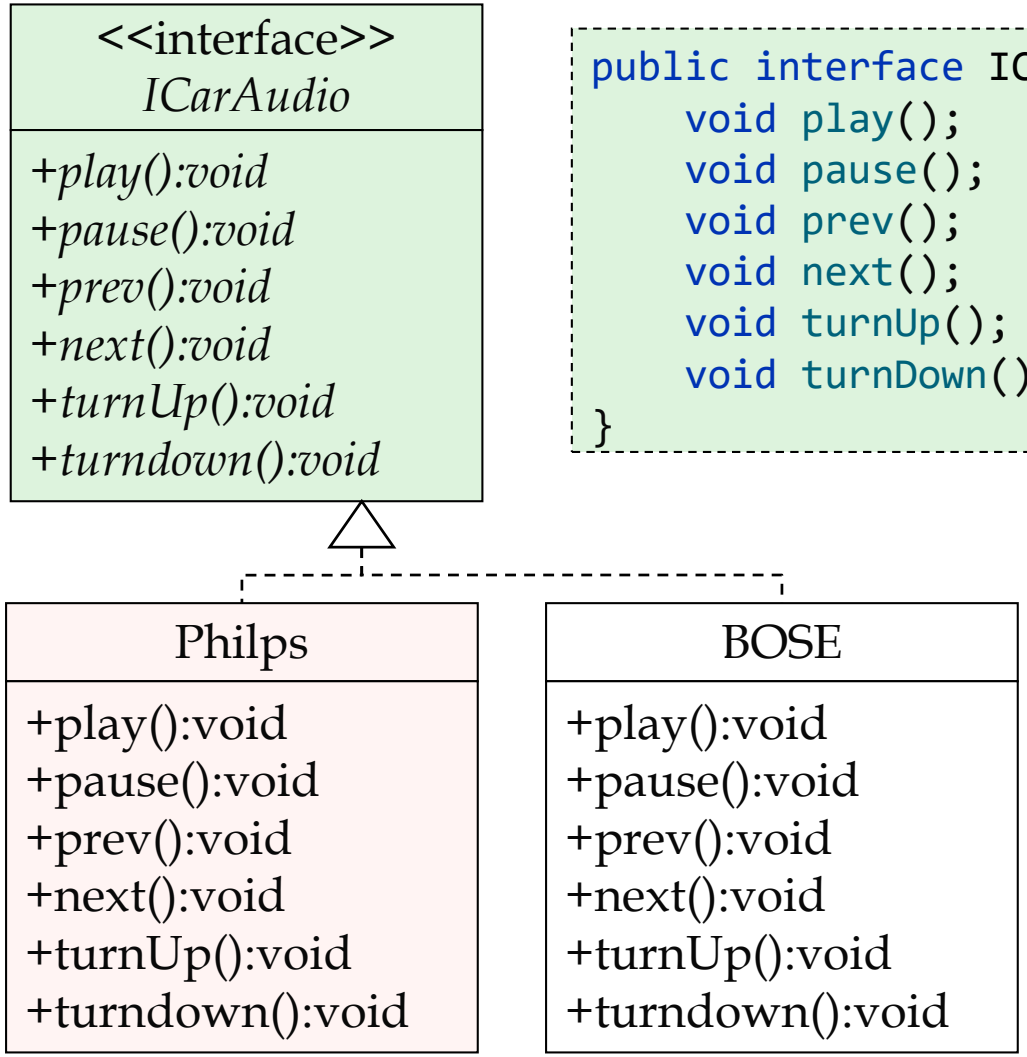
Car类通过构造方法注入具体厂家，即carAudio属性，因此没有 setter；

但可能读carAudio属性，因此需要添加carAudio属性对应的getter



- 协议：用 Java 中的接口表达
- 协议规定了各个子类必须实现的动作
- 即：厂商必须实现协议

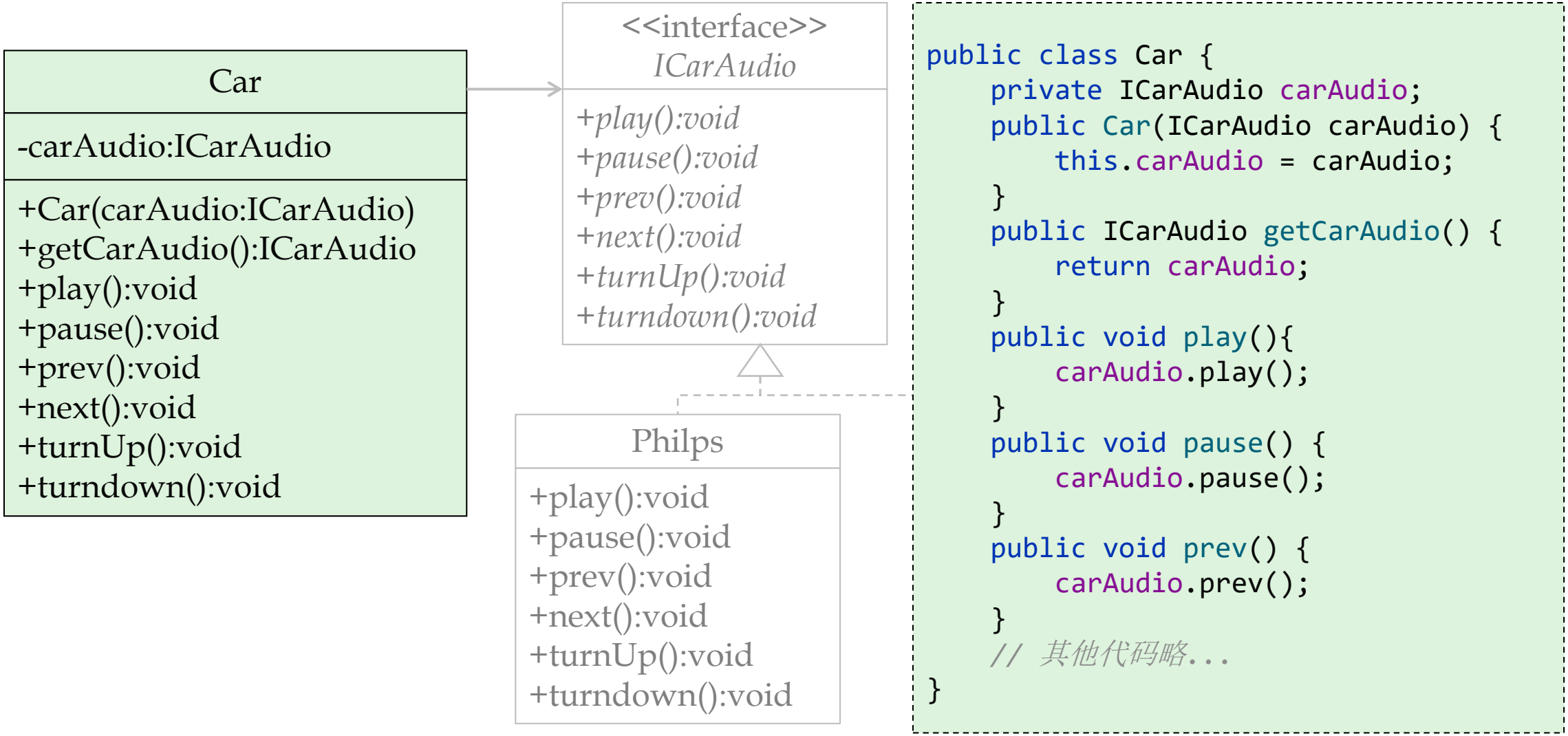
3.3 编写代码框架




```
public interface ICarAudio {
    void play();
    void pause();
    void prev();
    void next();
    void turnUp();
    void turnDown();
}
```

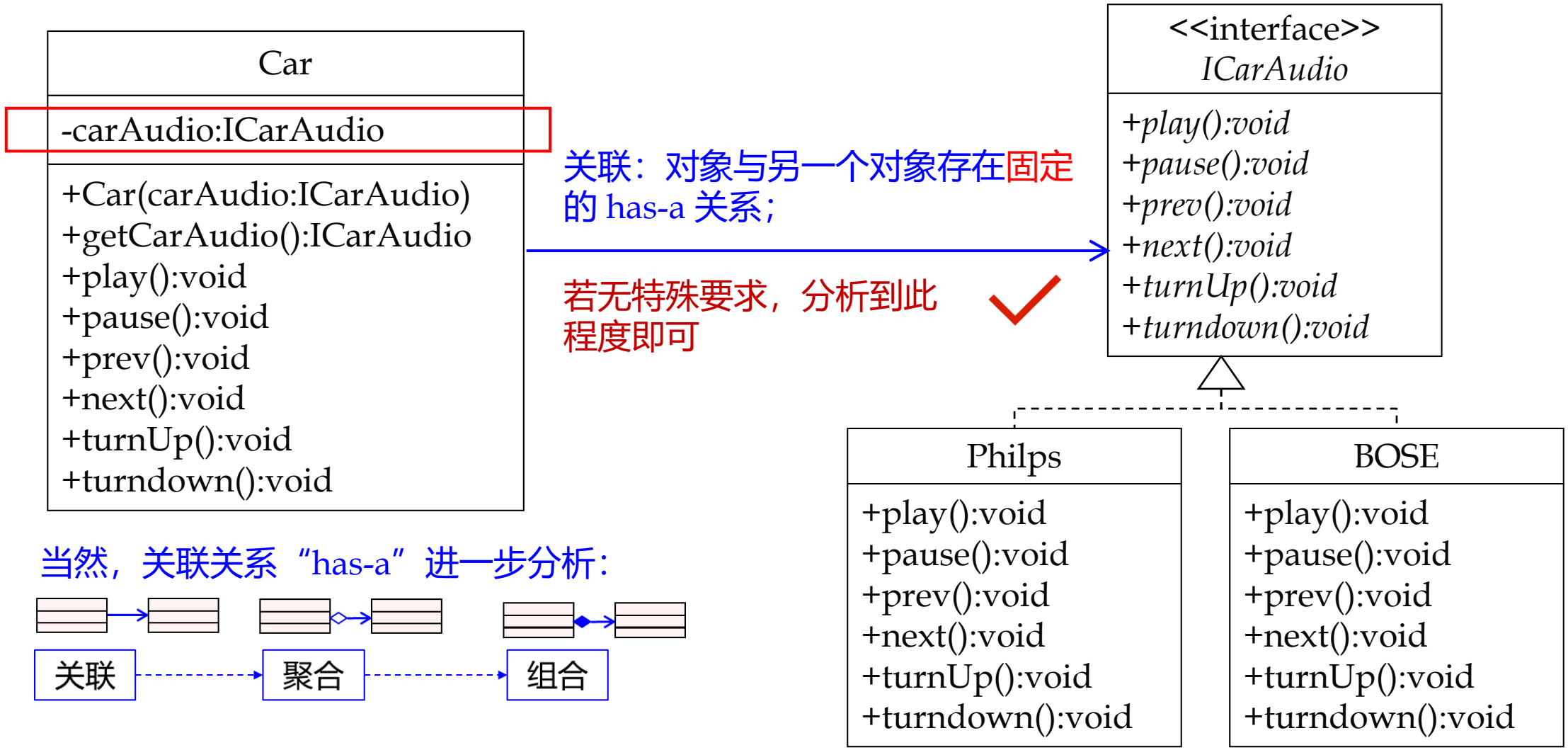
```
public  class Philps implements ICarAudio {
    @Override
    public void play() {
        // 具体代码实现
    }
    @Override
    public void pause() {
        // 具体代码实现
    }
    @Override
    public void prev() {
        // 具体代码实现
    }
    @Override
    public void next() {
        // 具体代码实现
    }
    @Override
    public void turnUp() {
        // 具体代码实现
    }
    @Override
    public void turnDown() {
        // 具体代码实现
    }
}
```

3.3 编写代码框架



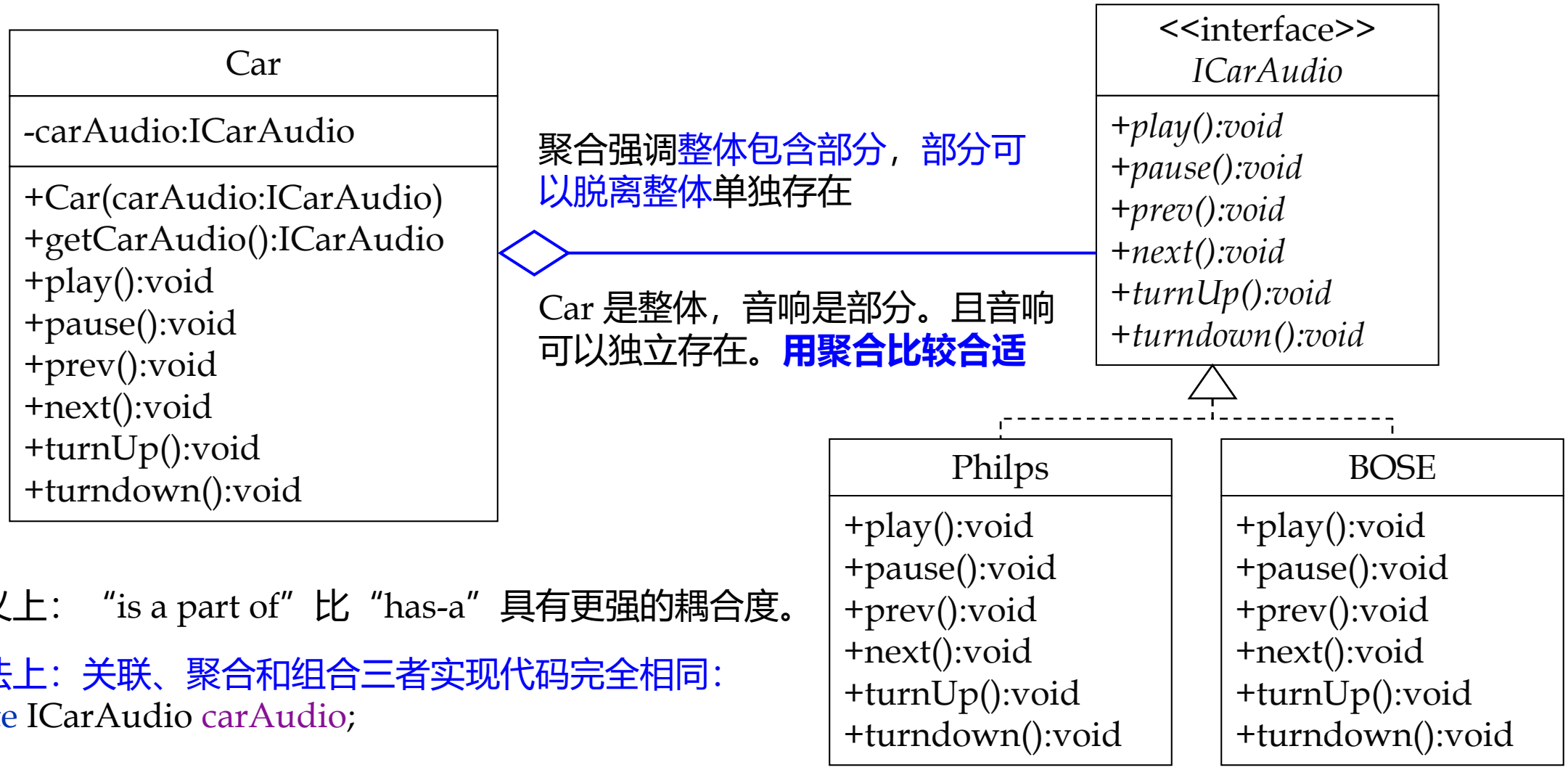

```java
public class Car {
    private ICarAudio carAudio;
    public Car(ICarAudio carAudio) {
        this.carAudio = carAudio;
    }
    public ICarAudio getCarAudio() {
        return carAudio;
    }
    public void play(){
        carAudio.play();
    }
    public void pause() {
        carAudio.pause();
    }
    public void prev() {
        carAudio.prev();
    }
    // 其他代码略...
}
```

3.4 类图重构-讨论：关联 vs 聚合



汽车和汽车音响是整体和部分的关系。　而表达整体与部分关系，需要用聚合和组合

3.4 类图重构-讨论：关联 vs 聚合

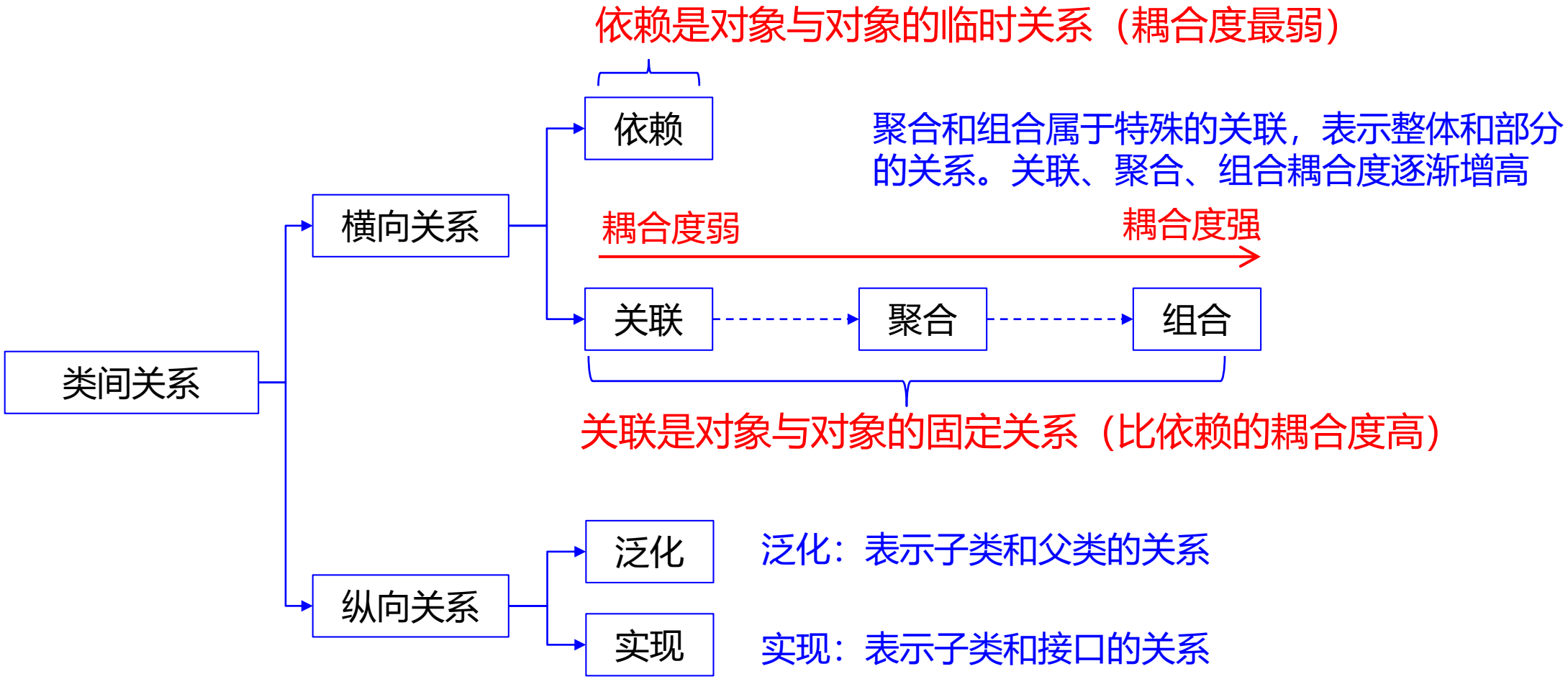


在语义上： “is a part of”比“has-a”具有更强的耦合度。

在语法上：关联、聚合和组合三者实现代码完全相同：

```
private ICarAudio carAudio;
```

本章总结



耦合强度：横向关系耦合度较低，纵向关系耦合度较高；依赖<关联<聚合<组合<泛化（继承）